FaultNet Network Resilience Testing using Controlled Fault Injection

BCSE308P - Computer Networks

PROJECT BASED COMPONENT REPORT

by

Pranav Suresh

24BCE2086

Soham Sameer Borikar

24BAI0142

School of Computer Science and Engineering



DECLARATION

We hereby declare that the report entitled "FaultNet Network Resilience Testing using Controlled Fault Injection" submitted by us, for the BCSE308P - Computer Networks to VIT is a record of Bonafide work carried out by us under the supervision of Dr. S. Ebenezer Juliet.

We further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for any other courses in this institute or any other institute or university.

Place: Vellore

Pranav Suresh

Soham Sameer Borikar

Signature of the Candidate(s)

TABLE OF CONTENTS

1. Introduction 4
1.1 Practical Applications 4
2. Problem Statement 4
3. Objectives 5
4. Literature Review and Existing System 6
5. Abstract 7
6. Workflow of the Proposed Project work and System Architecture 8
6.1 Dashboard Roles and Separation 9
6.2 Workbench Functions and Fault Mechanisms 9
6.3 Live Proxy Cockpit Functions 10
7. Contribution of Individual Team Member 11
8. Modules Explanation 11
9. Project Implementation 12
9.1 Experimental Setup with Hardware and Software Requirements 12
9.2 Test Cases 13
9.3 Results of Execution and Screenshots 14
9.4 Performance Analysis and Comparison of Methods 16
9.5 Critical Evaluation and Optimization 25
10. Results and Discussion 26
10.1 Findings and Inferences 26
10.2 Innovation Novelty and Evidence 26
11. Conclusion 26
12. Limitations and Future Enhancement 27
References 28

1. Introduction

Reliable communication requires more than a successful request under normal conditions. A service can return correct data while delivering an unacceptable response time, and a connection can end before its body is complete. Developers need controlled conditions that expose these differences and evidence that connects a configuration to its observed effect.

FaultNet is a local resilience-testing system with two complementary interfaces. Its experiment workbench runs a fixed HTTP workload through baseline, faulty and recovery phases. Its live proxy cockpit applies operator-selected effects to development requests. Wireshark independently observes the real packet exchange, while separate Cisco Packet Tracer labs examine reachability after a simulated router-link failure.

The central contribution is a reproducible connection between configuration, measurement and interpretation. The evaluation measures fault calibration, request completion, payload goodput, timeout tradeoffs and restoration. It also reports the cost of interception and the conditions under which the configured delivery rate is not attained. The project does not assume that a testing tool makes a connection faster; it makes failure behavior inspectable.

1.1 Practical Applications

Potential applications include checking a registration portal under delayed responses, examining a checkout interface after an HTTP 503 response, testing a download client under restricted delivery or disconnection, and exercising a frontend against controlled JSON responses. A development client must actually route its requests through the live entry for these controls to act. The implemented experiments use synthetic data rather than real customer transactions.

For teaching, the workbench provides comparable phase observations and saved evidence. Packet analysis connects an HTTP result with packet timing, and the routing labs illustrate how an alternate physical path becomes useful only when a routing method selects it. These activities link application behavior, transport evidence and network-layer reachability without conflating them.

2. Problem Statement

Ordinary application testing often establishes that a response is correct without measuring how the same workload behaves when communication is slow, rate-limited or interrupted. Uncontrolled repetitions can change the timeout, payload or request count and produce misleading comparisons. A dashboard can also hide failures if it presents only successful timing samples.

The project addresses four questions: whether the configured faults produce measurable changes; whether deadline choices affect completion; whether fault clearance restores successful responses; and whether the supporting packet and routing evidence is consistent with the claimed behavior. Measurements must preserve failed outcomes, sample counts and units as well as successful-response statistics.

3. Objectives

- Establish a fixed 65,536-byte workload and compare its direct and proxied execution.
- Quantify added response time and its deviation from configured delays.
- Compare measured payload goodput with configured response-body restrictions.
- Measure request success, failure duration and recovery evidence without replacing missing successful timings by zero.
- Evaluate the completion-versus-waiting tradeoff while changing only the client timeout.
- Demonstrate manual live-proxy controls, independent packet evidence and static/RIP/OSPF link-failure behavior.
- Preserve settings and observations so that report tables and figures can be reproduced.

4. Literature Review and Existing System

HTTP semantics distinguish the meaning of a response status from the success of transmitting a complete message [1]. This matters when comparing a valid HTTP 503 response with a request whose connection closes before completion. A complete transport exchange can carry an application error, while an interrupted exchange may have no usable status or payload. FaultNet records these cases separately.

Node.js supplies the HTTP client/server facilities and a high-resolution elapsed-time clock used by the implementation [2]. These facilities permit real loopback exchanges without a framework dependency. Elapsed timing and UTC timestamps serve different purposes: the former measures duration, while the latter associates observations with capture evidence.

Toxiproxy is an established TCP proxy for testing network and system conditions [3]. Its existence shows that proxy-based fault injection is an established technique. FaultNet's contribution is its educational HTTP workflow, integrated phase statistics, saved records and separate live cockpit, rather than a claim to have invented latency or bandwidth injection. This report does not benchmark Toxiproxy or claim a speed advantage over it.

Wireshark reconstructs protocol messages from underlying packet segments. Reassembled data is displayed on the final packet of a reconstructed unit, which can differ from the first arrival of response data [4]. Accurate interpretation therefore separates first-response timing from complete-response timing. Packet Tracer routing labs supply a different form of evidence: RIPv2 uses distance-vector routing with hop-count metrics [5], while OSPF maintains link-state information and derives paths using costs [6]. Their metric values are not directly interchangeable.

Table 1. Roles of existing approaches and the project

Approach	Useful evidence	Relationship to FaultNet
Ordinary request / ping	Current completion or reachability	Does not independently impose the selected HTTP fault.
Toxiproxy	Established TCP fault injection	Related prior tool; no comparative execution-speed claim.
Wireshark	Packets, protocol contents and timings	Independent observer of FaultNet traffic.
Packet Tracer	Simulated routing and link-failure behavior	Separate network-layer demonstration.
FaultNet	Configured faults, phase statistics and saved records	Combines controlled HTTP execution with inspectable evidence.

5. Abstract

FaultNet is a local HTTP resilience-testing system that combines automated experiments with manual development-proxy controls. The workbench coordinates baseline, faulty and recovery phases against a fixed 64 KiB response, validates observations and saves JSON evidence. The evaluation comprises 100 workbench runs with 1300 phase observations, 100 direct/proxy calibration requests and 80 live-proxy samples. Ten repeated runs are used per workbench configuration. The 1,500 ms delay condition adds a median 1,506.52 ms relative to matching baselines. At 16 KiB/s, observed payload goodput is 15.96 KiB/s, while the 128 KiB/s condition attains 61.81% of its configured rate. Increasing timeout from 500 to 2,000 ms under a fixed 1,000 ms delay changes faulty completion from 0% to 100%. Packet timings provide independent traffic evidence. Reviewed routing cases show that RIP and OSPF use the alternate path after the tested direct-link failure, whereas the static control lacks a backup route. The findings support controlled local fault testing and expose platform-dependent pacing limitations.

6. Workflow of the Proposed Project work and System Architecture

A single Node.js process owns four loopback listeners. Port 3000 serves both interfaces and their protected control APIs. The fixed demo on port 3001 returns HTTP 200 and exactly 65,536 synthetic F bytes for GET /payload. The experiment proxy on port 3002 applies the workbench fault. The separate live proxy on port 8080 forwards development traffic under its applied configuration.

Table 2. Listener responsibilities

Component	Port	Function
Control server	3000	Dashboard assets, settings, readiness, records, exports and live control.
Demo server	3001	Fixed response contract for repeatable experiments.
Experiment proxy	3002	One selected delay, rate restriction or interruption per run.
Live proxy	8080	Independent configurable HTTP/HTTPS development forwarding.

The measured workbench path is Node probe → experiment proxy → demo server; the body returns through the proxy to the probe. The runner assigns phases, clears faults and saves observations. The browser retrieves measurements through the control API and renders charts; it neither generates the measured duration nor directly reads result files. Control traffic is kept separate from the faulted data path.

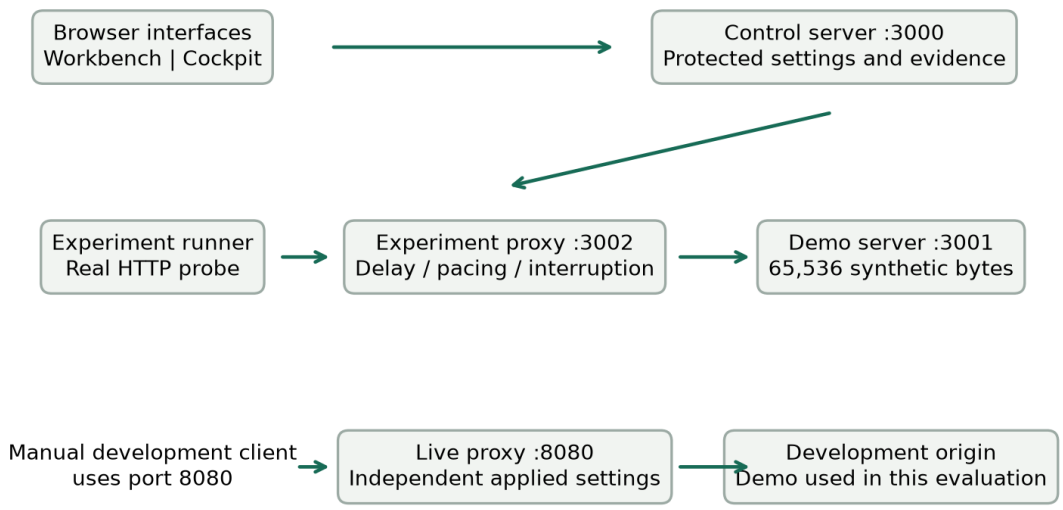


Figure 1. Control and data paths in FaultNet. The workbench and live proxy have independent fault configurations; replies return through the same proxy path.

The workbench validates a request, saves its initial record, runs baseline probes, activates the selected fault, runs faulty probes, clears the fault, checks recovery and saves the terminal record. One experiment is active at a time. Stop cancels active work and clears held operations. If a process ends unexpectedly, a previously active persisted record is marked interrupted on restart rather than silently completed.

6.1 Dashboard Roles and Separation

Table 3. Why the project uses two dashboards

Aspect	Experiment workbench	Live proxy cockpit
Question	How does this fault alter a controlled workload?	How do individual development requests behave under applied effects?
Target	Fixed 64 KiB demo	Validated development origins; demo by default
Sequence	Automatic baseline/fault/recovery	Operator applies settings, sends traffic and resets
Persistence	Validated local JSON and JSON/CSV exports	Latest 50 finished requests in process memory
Measurements	Phase counts, distributions and comparisons	Per-request status, duration, bytes and applied effects

Separate configurations protect reproducible measurements from exploratory edits. The workbench constrains the workload and comparison protocol; the cockpit provides broader manual combinations. Changing one dashboard does not change the other proxy's fault configuration. Neither interface automatically repairs a failing application.

6.2 Workbench Functions and Fault Mechanisms

Six editable presets cover a healthy baseline, mild/severe delay, constrained bandwidth, timeout stress and complete HTTP outage. A preset fills settings without starting traffic. Start begins the experiment; Stop requests cancellation; saved-run reuse copies a configuration without modifying its evidence. Auto or explicit phase selection controls the displayed analysis. Protocol-compatible saved runs can be compared, and records can be exported as JSON or CSV.

Table 4. The three workbench fault mechanisms

Mechanism	Implementation action	Measured consequence
Delay	Await a timer before forwarding	Additional complete-response time; possible client timeout.
Bandwidth	Schedule 1 KiB body chunks at rate-based deadlines	Longer transfer and lower payload goodput.
Interruption	Destroy the faulty client connection before forwarding	No complete successful HTTP payload; subsequent recovery checked after clearance.

The timer wait is asynchronous and does not busy-wait. Bandwidth scheduling honors downstream backpressure and buffers only the known payload. These operations alter actual proxied HTTP exchanges, not the laptop's physical bandwidth or an IP packet-loss percentage. Timeout stress is an existing delay mechanism paired with a shorter client deadline.

Each probe succeeds only after HTTP 200, all 65,536 bytes and the expected payload contents arrive before timeout. Mean, median and p95 use successful samples. Eligible requests exclude

user cancellation, and unavailable successful timing remains null. Charts present all three phase medians, attempt durations, successful-response distributions and successful/failed/cancelled counts.

6.3 Live Proxy Cockpit Functions

Unified routing forwards to one origin; split routing selects frontend or backend origins using path prefixes. The client must use the port-8080 entry for effects to apply. Presets/sliders change a draft, Apply changes activates it for new requests, and Reload applied retrieves current settings. Reset removes effects and rules, discards drafts and cancels live work while retaining the applied targets and routes.

Table 5. Live fault and response controls

Control	Purpose
Request/response delay and jitter	Vary forwarding and response waits; jitter applies in both directions and clamps negative waits to zero.
Upload/download limits	Pace body delivery in KiB/s; zero means unlimited.
Timeout / stall / disconnect	Delay headers, hold a body, or interrupt body delivery.
Status injection	Return a selected 400–599 status for a path prefix at a configured probability.
Header controls	Remove CORS/cache headers or mismatch Content-Type.
JSON mocks	Return local JSON without contacting upstream; selected status rules have precedence.

Send sample performs a GET with a five-second client ceiling. The monitor provides status, duration, bytes, outcome, effect labels, filtering and aggregate counts for the latest 50 finished requests. Clear monitor affects that window only. Live configuration and monitoring survive a page refresh but reset with the process. Named mobile profiles are illustrative settings, not carrier measurements.

7. Contribution of Individual Team Member

Table 6. Division of project responsibilities

Member	Registration number	Project responsibilities
Pranav Suresh	24BCE2086	Application requirements and iteration, experiment demonstration, interpretation of workbench/cockpit behavior, and report coordination.
Soham Sameer Borikar	24BAI0142	Packet Tracer lab preparation and routing demonstration; coordination of the separately maintained lab work.

The application and routing components form one demonstration with distinct evidence paths. Both members must be able to explain the fault settings, actual observations, routing changes and limitations of the conclusions. The source repository preserves application context and the separately reviewed lab history.

8. Modules Explanation

Table 7. Important code modules

Important module	Responsibility
src/app.js and src/main.js	Dependency wiring, listener startup and orderly shutdown.
src/control/server.js	API routing for experiments, records, export and live controls.
src/control/validation.js and security.js	Input bounds, allowed shapes, Host/Origin checks and capability protection.
src/experiments/runner.js	Phase coordination, cancellation, deadline, recovery and saves.
src/experiments/probe.js	HTTP request, elapsed timer, payload/status verification and outcomes.
src/experiments/metrics.js	Phase counts, mean/median/p95 and payload goodput.
src/proxy/server.js	Fixed-target delay, body pacing and interruption.
src/live/server.js and config.js	Independent forwarding, validated targets and live effects.
src/storage/records.js and export.js	Record validation, safe persistence and JSON/CSV export.
public/app.js, results.js and charts.js	Workbench events, derived views, comparisons and SVG visualizations.
public/live.js and live-model.js	Applied/draft controls, presets and live traffic presentation.

9. Project Implementation

9.1 Experimental Setup with Hardware and Software Requirements

The application evaluation used Windows_NT build 10.0.26300, an 12th Gen Intel(R) Core(TM) i7-12650H processor and 23.7 GiB of visible memory. The measured application runtime was v22.23.1. The application uses Node built-in HTTP, timing, filesystem and test facilities with plain HTML/CSS/JavaScript and no external runtime packages. Analysis tooling is separate from application dependencies.

The application source baseline is commit 313822e in the project repository. Measurement and report scripts are maintained separately from the application modules. Repeating an evaluation must preserve the recorded settings and runtime; a different laptop or source revision can legitimately produce different timings.

All measured traffic used the numeric loopback address. HTTP probes disabled connection pooling, verified the fixed payload, and measured the entire exchange with performance.now(). UTC starts identify records for packet matching. Background desktop applications and normal operating-system scheduling remained present; the results describe this local environment rather than an isolated hardware benchmark.

Project repository: https://github.com/Pranav-Suresh1024/FaultNet

Table 8. Measurement protocol and reproducibility controls

Protocol choice	Value or procedure
Repeated blocks	10 runs per configuration; cyclic configuration order across blocks.
Workload	65,536 F bytes; five baseline and five faulty probes per run.
Interval	250 ms between probes within a phase.
Default deadline	5,000 ms; timeout study changes only this to 500 or 2,000 ms.
Recovery	Three consecutive successful responses within 15,000 ms after clearance.
Overhead test	50 direct and 50 fault-free proxied requests, grouped in ten paired blocks; three warm-up pairs excluded.
Live study	Eight configurations, ten sequential samples each; five-second sample ceiling.
Packet capture	Npcap loopback; TCP 3001, 3002 and 8080; dashboard control excluded.

The comparison budget is 2 × count × (timeout + interval) + 15,000 ms. Five probes, a 5,000 ms timeout and 250 ms interval give a conservative 67,500 ms bound, below 120,000 ms. This is a safety bound, not a duration prediction. The timeout pair retains the same payload, delay, count and interval; its different deadlines intentionally make it incompatible with the dashboard's automatic cross-run guard.

9.2 Test Cases

Table 9. Experimental cases and hypotheses

Condition	Controlled variable	Expected observation
Delay calibration	0, 150, 500, 1,500 ms	Added timing follows the injected wait with measurable overhead.
Bandwidth calibration	16, 32, 128 KiB/s	Ideal 64 KiB delivery is 4, 2 and 0.5 seconds; actual rate is measured.
Connection interruption	Close each faulty connection	Faulty requests fail to complete, followed by successful clearance/recovery.
Timeout tradeoff	1,000 ms delay; 500 vs 2,000 ms timeout	Completion may improve when the deadline exceeds the delay.
Live behavior	Normal, delay, jitter, rate, 503, disconnect, stall, mock	Each request exhibits the applied effect with honest status/bytes/outcome.
Routing link failure	R1 G0/0 disabled	Static control lacks backup; dynamic labs can select R3.

A completed experiment is not automatically a successful faulty workload: the interruption and short-timeout conditions can complete the experiment while recording all faulty requests as failures. No cancelled request was reclassified as a successful timing sample. Run-level record validation checks structure, summaries, ordering and recovery chronology before analysis.

Within each run, added delay is faulty median minus baseline median. The calibration table reports the median of these ten paired differences. The exploratory bootstrap interval resamples run-level differences, not individual requests, with a fixed seed and 3,000 resamples. It describes repeated variation on this machine; it is not a population guarantee. Pooled p95 is labelled descriptive for 50 attempts per configuration.

Availability (%) equals 100 × successful attempts / eligible attempts; user cancellations are excluded from eligibility. For a successful fixed response, payload goodput (KiB/s) equals 64 / (elapsed milliseconds / 1,000); the table reports the arithmetic mean of these per-request rates. The nearest-rank p95 uses position ceiling(0.95 × N) in sorted successful timings. These definitions preserve the distinction between slow success, failed completion and missing successful latency.

The configured automated test command passed all 61 tests with zero failures or skipped tests. Coverage includes fault behavior, cancellation and cleanup, recovery, persistence/export validation, control security, live proxy behavior and frontend calculations. This verifies the tested paths; it does not establish correctness under every workload or replace the separate packet and simulator evidence.

9.3 Results of Execution and Screenshots

The completed dataset contains 100 workbench runs and 1300 phase observations, plus 100 overhead requests and 80 live samples. Each workbench condition has ten runs and 50 faulty attempts. The following tables report measurements, while the comparison plots explain their relationship to configuration. The workbench interface figure is a representative reviewed screenshot; its displayed example values are separate from the repeated-study aggregates.

Record quality is reported separately from request success. There were 101 attempted workbench runs: 100 supplied validated terminal records and 1 failed record validation. In the rejected bandwidth trial, UTC recovery duration and elapsed-clock duration differed by 2.38 ms, above the validator's 2 ms tolerance. The retained rejected record is excluded from timing aggregation and the condition is repeated to preserve the ten-run design. Thus 100 accepted runs do not imply that every attempted record was saved successfully.

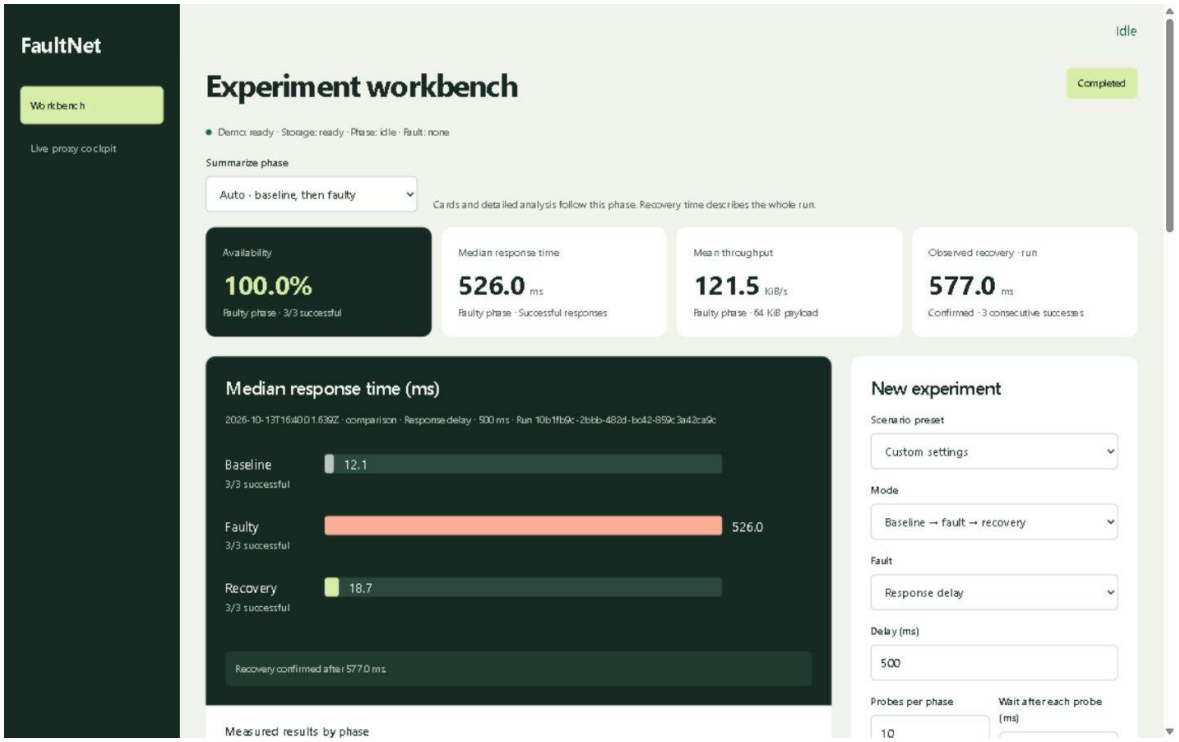


Figure 2. Representative workbench interface showing settings, phase comparisons, successful timing and recovery evidence. Displayed example values are not the repeated-study aggregate.

The workbench already exposes the measurements required for this evaluation through validated records and exports. No production dashboard or backend change was necessary. Repeated-run summaries and research figures are derived from those observations by the report analysis scripts, preserving the existing measurement contract.

Table 10. Faulty-phase results across repeated workbench trials

Condition	Success	Median ms	p95 ms	Goodput KiB/s
delay-0	50/50	17.81	20.65	4,822.55
delay-150	50/50	160.72	167.87	397.47
delay-500	50/50	507.53	517.91	125.56
delay-1500	50/50	1,510.35	1,522.01	42.35
bandwidth-16	50/50	4,010.29	4,017.72	15.96
bandwidth-32	50/50	2,012.64	2,017.88	31.81
bandwidth-128	50/50	819.62	929.94	79.12
interruption	0/50	Not applicable	Not applicable	Not applicable
timeout-500	0/50	Not applicable	Not applicable	Not applicable
timeout-2000	50/50	1,010.22	1,018.53	63.32

A missing successful median or goodput is not zero: interruption and deadline failures have no successful full-response sample. Their attempted durations are analysed separately. Bandwidth and delay both reduce whole-request goodput, but their mechanisms differ: one paces the body and the other waits before forwarding.

9.4 Performance Analysis and Comparison of Methods

9.4.1 Delay Calibration and Distribution

Table 11. Configured delay against paired added response time

Delay ms	Paired added ms	Error %	Bootstrap interval ms
150	156.55	4.37	155.93 to 157.89
500	503.41	0.68	502.09 to 506.54
1500	1,506.52	0.43	1,503.80 to 1,508.07

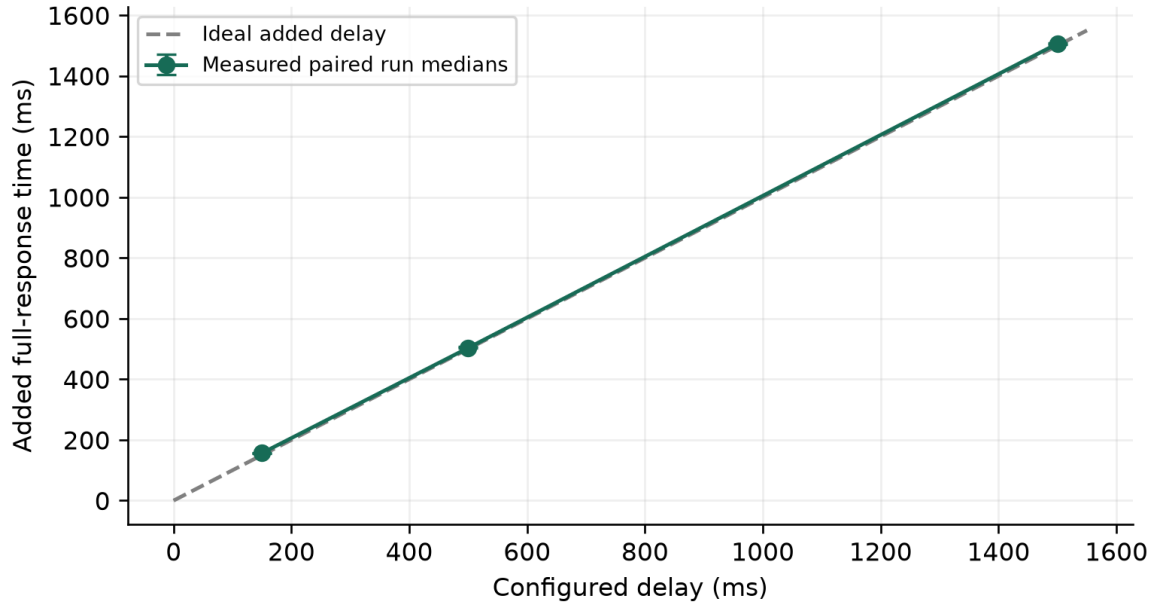


Figure 3. Configured delay versus paired measured increase. Error bars are exploratory bootstrap intervals across ten runs.

The 1,500 ms condition produced an added median of 1,506.52 ms, an error of 0.43%. Absolute overhead becomes proportionally more visible at small configured delays. The zero-delay condition is an implementation control: the code still dispatches an asynchronous timer, so a configured zero does not establish zero timer or interception overhead.

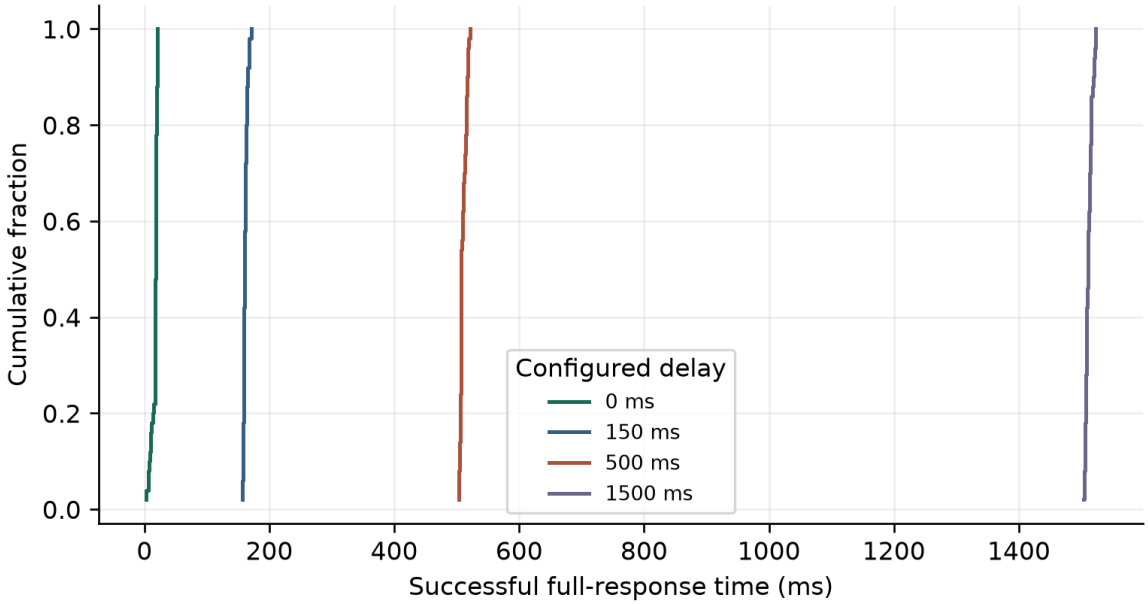


Figure 4. Empirical distributions of successful faulty response times; 50 observations per configured delay. Curves show separation and within-condition variation.

Distribution separation shows that the configured wait changes actual response behavior, rather than only changing a summary label. A percentile based on 50 successful samples remains descriptive. It should be read alongside the number of successes, failure outcomes and the repeated-run spread.

9.4.2 Bandwidth Shaping and Rate Attainment

Table 12. Configured response rate compared with measured payload goodput

Limit KiB/s	Ideal ms	Median ms	Goodput KiB/s	Attainment %
16	4,000.00	4,010.29	15.96	99.75
32	2,000.00	2,012.64	31.81	99.40
128	500.00	819.62	79.12	61.81

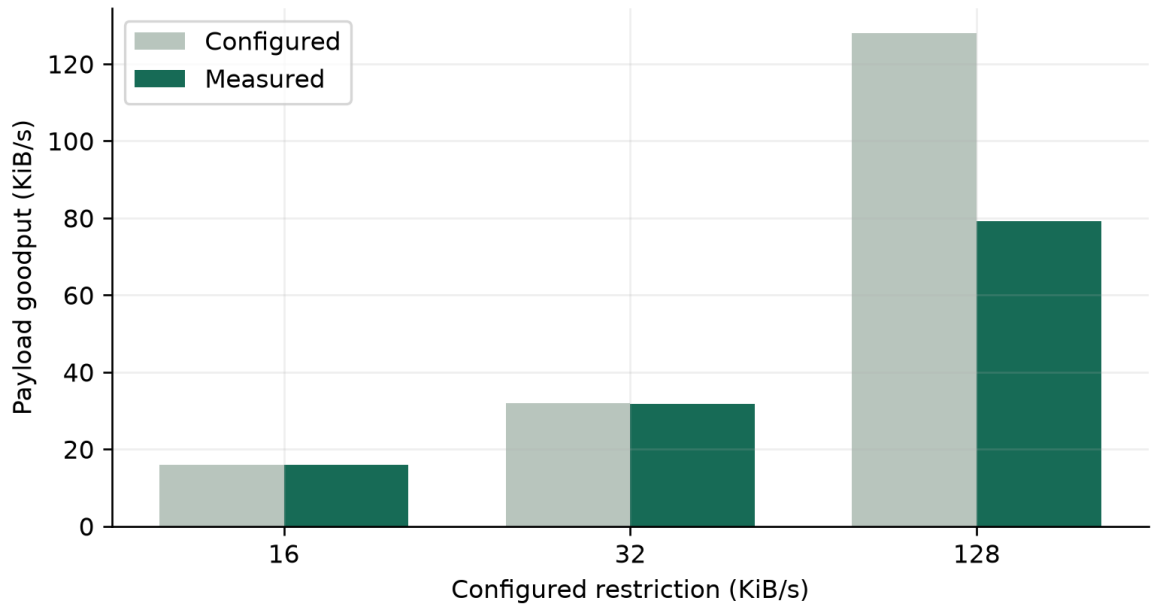


Figure 5. Measured whole-request goodput versus configured restriction for the same 64 KiB payload.

The 16 KiB/s condition attained 99.75% of its configured rate. The 128 KiB/s condition attained 61.81%, so the implementation should be interpreted as a delivery cap rather than a guarantee of exact target throughput. The 1 KiB schedule has nominal chunk intervals of 62.5, 31.25 and 7.8125 ms at 16, 32 and 128 KiB/s respectively. At short waits, timer dispatch and local scheduling can consume a larger fraction of the interval. This is a plausible explanation consistent with the mechanism; timer causes were not independently isolated.

This comparison is useful even where calibration is weaker: it identifies the operating range where the simple pacing approach follows its configuration closely. The report retains the high-rate result instead of silently discarding it or widening an accuracy claim.

9.4.3 Timeout Tradeoff and Failure Outcomes

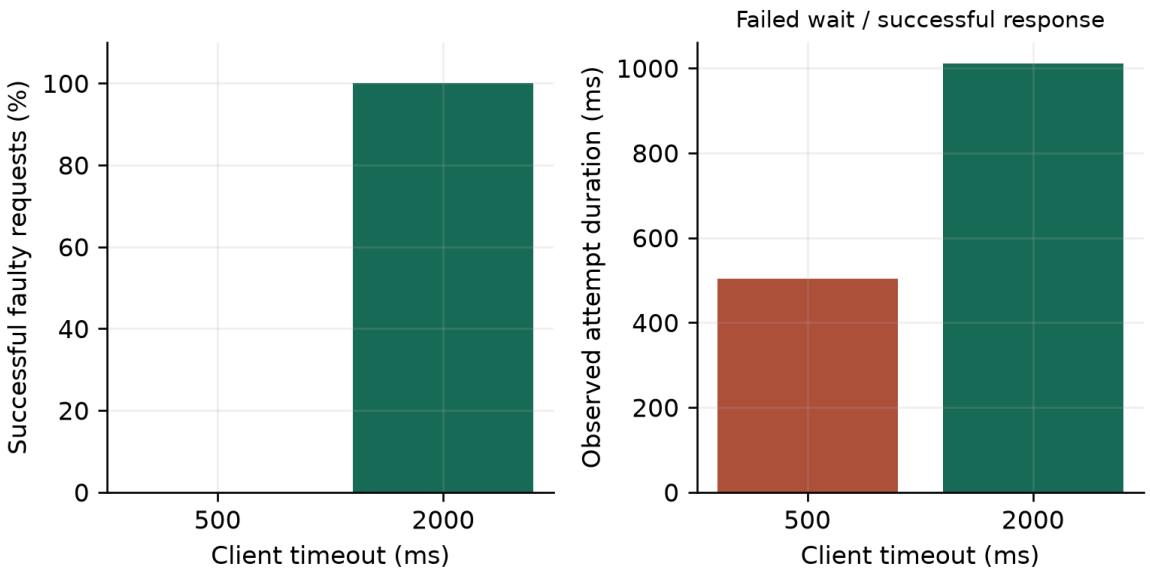


Figure 6. Fixed 1,000 ms injected delay with two client deadlines. The duration panel distinguishes failed waiting time from successful response latency.

At a 500 ms timeout, 0 of 50 faulty requests succeeded and the median failed-attempt duration was 503.88 ms. At 2,000 ms, 50 of 50 succeeded with a successful median of 1,010.22 ms. The observed completion difference is 100 percentage points. The delay itself was unchanged. Longer waiting improves tolerance in this case, but also postpones failure detection when a service will not recover.

The interruption condition produced 0 successful faulty responses out of 50. This is an application-level completion result; it is not an estimate of random IP packet loss, nor does it imply that every connection necessarily used a TCP reset flag.

9.4.4 Interception Cost and Recovery

Across 50 direct and 50 fault-free proxied requests, pooled medians were 2.37 and 5.08 ms. The median paired block difference was 2.32 ms, with an exploratory bootstrap interval from 1.39 to 3.03 ms. This quantifies instrumentation cost on the fixed local workload. It is not a web-scale throughput measurement.

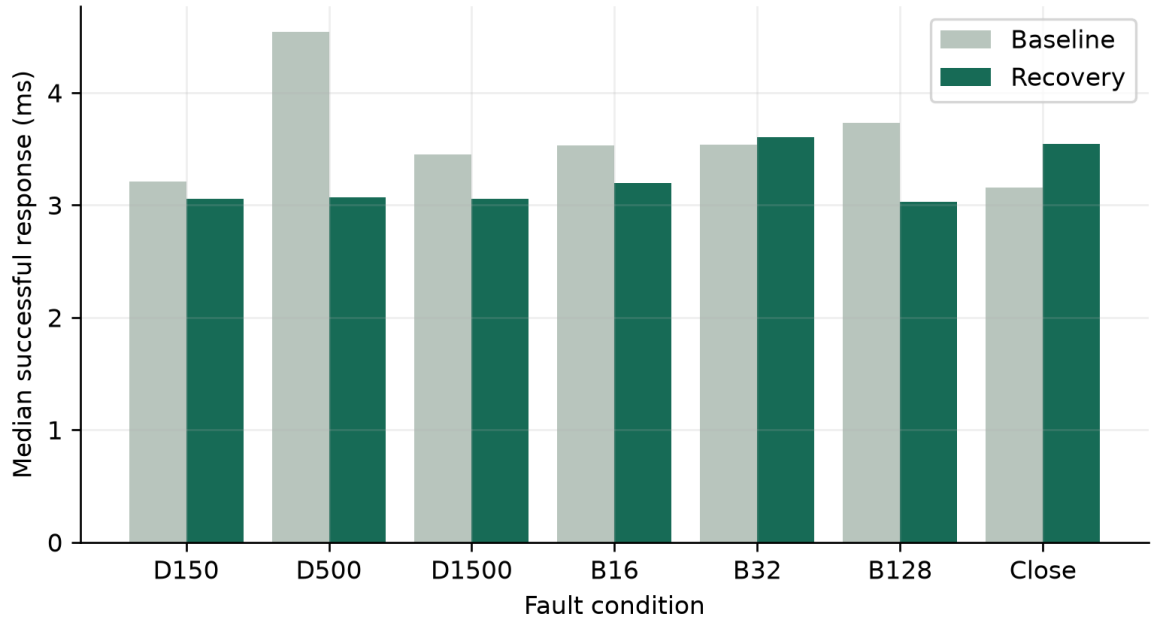


Figure 7. Baseline and recovered successful response times for each condition. Recovery phase medians and confirmation duration are distinct metrics.

Table 13. Restoration evidence across repeated runs

Condition	Confirmed runs	Confirmation median ms	Recovery minus baseline ms
delay-1500	10/10	531.10	-0.89
bandwidth-16	10/10	530.77	0.40
bandwidth-32	10/10	532.55	0.07
bandwidth-128	10/10	529.61	-0.50
interruption	10/10	536.42	0.14
timeout-500	10/10	532.42	0.24

With three required successes and 250 ms between recovery attempts, two waits alone contribute approximately 500 ms. Confirmation therefore includes observation policy and scheduling, not just fault removal. The recovery residual compares successful response behavior after clearance with that run's baseline and is useful alongside the confirmation flag.

9.4.5 Live Proxy Evaluation

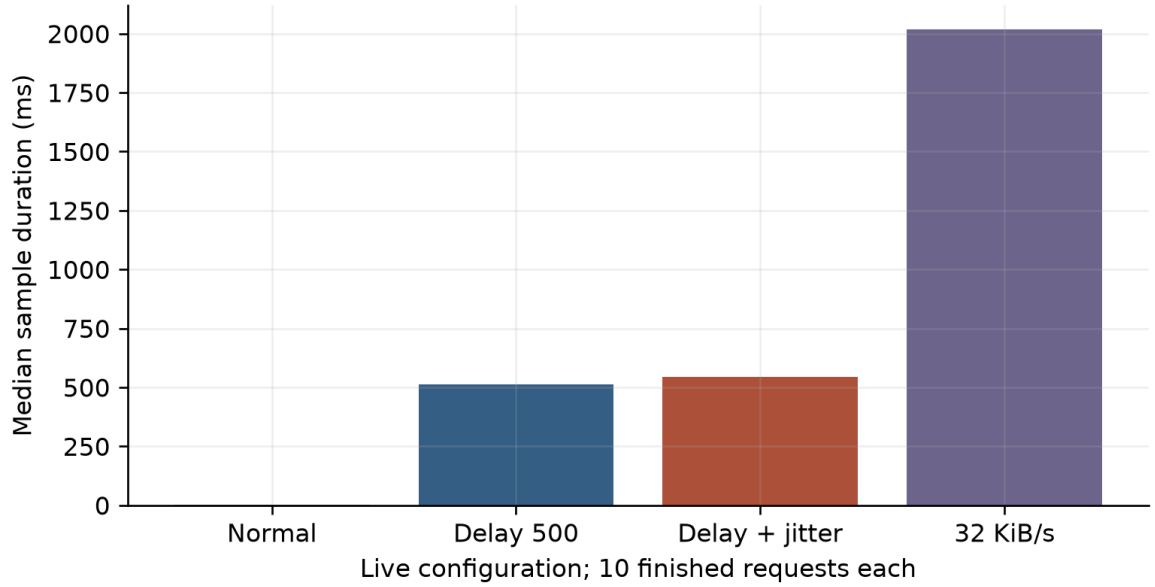


Figure 8. Live sample durations under normal, delayed, jittered and rate-restricted configurations; ten requests per condition.

Table 14. Manual live-proxy sample outcomes

Live condition	Completed	HTTP 200	Median ms	Median bytes
normal	10/10	10/10	2.80	65,536
delay-500	10/10	10/10	512.10	65,536
jitter-100	10/10	10/10	544.25	65,536
download-32	10/10	10/10	2,018.00	65,536
http-503	10/10	0/10	2.20	46
disconnect	0/10	0/10	32.25	16,384
stalled-body	0/10	0/10	5,012.25	0
mock	10/10	10/10	2.60	32

The fixed-delay condition had a sample standard deviation of 6.54 ms, compared with 57.66 ms under configured jitter. Jitter applies to both request and response waits, including the request direction with base zero, where negative variation is clamped. Consequently the jitter condition is not assumed to have exactly the same mean as fixed response delay alone.

The 100% HTTP 503 rule returned that status in 10/10 samples. Those samples can be completed HTTP exchanges while still reporting an application error. Stalled-body samples can receive headers before the five-second sample ceiling expires, so a recorded status 200 alone is insufficient to establish success. Body bytes and completion outcome must also be inspected. The final reset check returned HTTP 200 with 65,536 bytes.

9.4.6 Packet Evidence and Timing Correlation

The filtered loopback capture contains 55,392 packets; 0 packets match the dashboard control port. Complete HTTP reconstruction on port 3002 enables stream-specific timing checks. Probe starts are matched to GET timestamps and stream endpoints, with first response data and completed reconstruction considered separately. The capture began after the overhead warm-up sequence, so it is used only for matching covered experiment requests.

Table 15. Packet-to-probe correlation for representative covered runs

Condition	Matched streams	Median HTTP time ms	Median probe time ms
delay-500	5	504.98	507.62
delay-1500	5	1,507.79	1,510.29
bandwidth-32	5	2,010.28	2,011.82

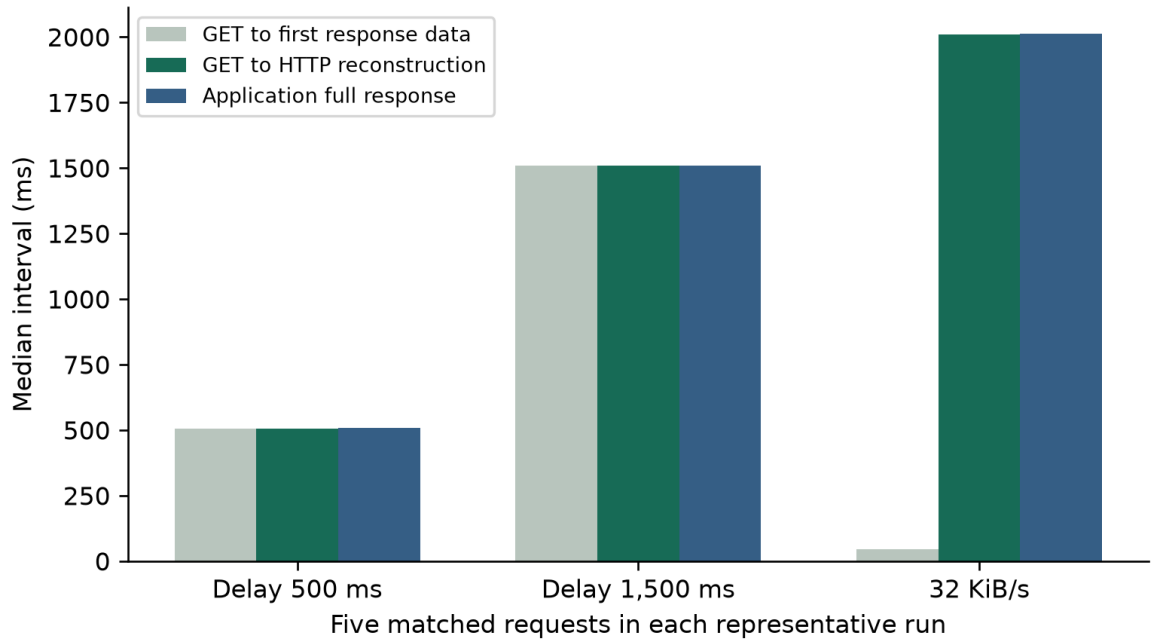


Figure 9. Measured request and response timing extracted with TShark from the loopback capture. This is an analytical packet view, not a simulated Wireshark screenshot.

Wireshark may label the final reassembly packet as the HTTP response even though response bytes began in an earlier segment [4]. Packet-derived intervals and application elapsed time therefore need not be numerically identical. Packet evidence is used as an independent consistency check, not as a reason to relabel application elapsed time as TCP acknowledgement RTT. Capture loss, if present, describes recording loss rather than the injected HTTP failure rate.

9.4.7 Routing Comparison in Packet Tracer

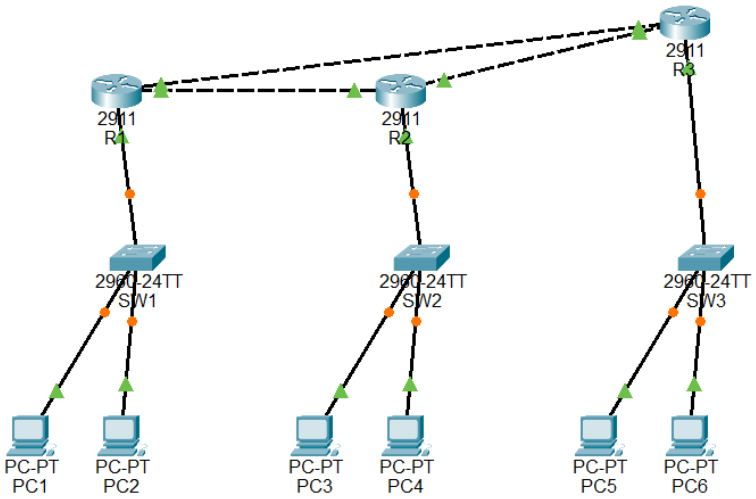


Figure 10. Reviewed Packet Tracer topology with three routers, three switches and six PCs. The router triangle offers an alternate R1–R3–R2 path.

The reviewed lab workload sends PC1 (192.168.0.2) to PC3 (192.168.1.2). R1 GigabitEthernet0/0 is disabled to remove the direct R1–R2 path. Stable ping batches and the R1 route table are inspected after convergence, and the interface is restored afterward. These simulator observations are a separately reviewed evidence set, not repetitions of the HTTP experiment dataset.

Table 16. Settled routing observations under the same direct-link failure

Method	Settled baseline	Direct link disabled	Alternate-route evidence
Static without backup	4/4 replies	0/4 replies	No configured alternate route for the destination LAN.
RIPv2	4/4 replies	4/4 replies	R 192.168.1.0/24 via 10.0.0.9, G0/1; [120/2].
OSPF	4/4 replies	4/4 replies	O 192.168.1.0/24 via 10.0.0.9, G0/1; [110/3].

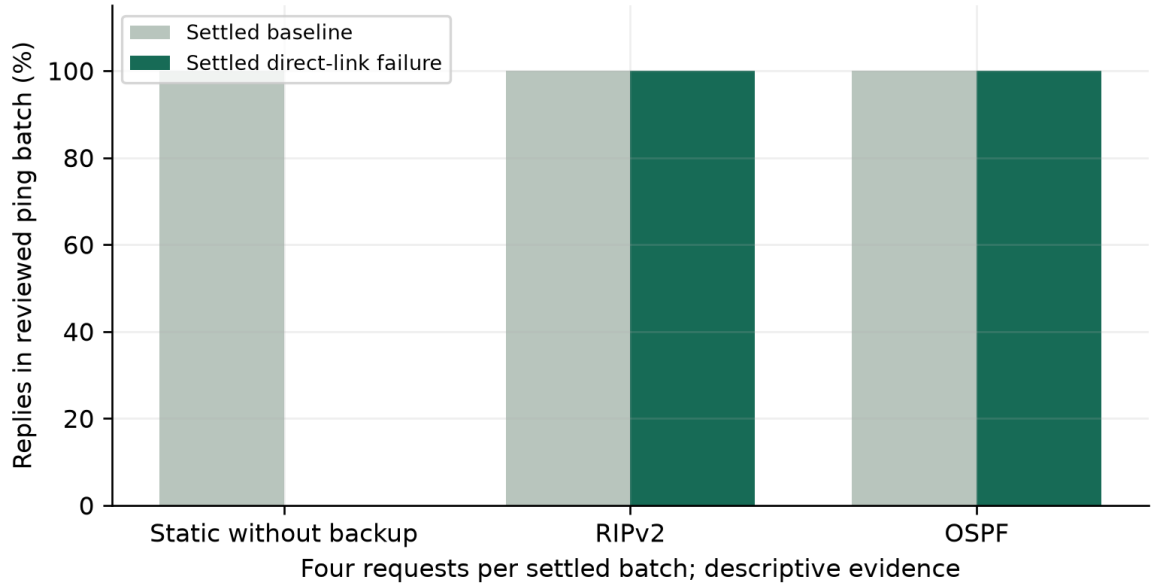


Figure 11. Descriptive reachability comparison of reviewed four-ping batches. Bars are observations of this topology, not reliability estimates for each protocol.

Dynamic routing preserved reachability in the settled fault state of this topology, while the deliberately unprotected static configuration did not. Static routing can be configured with backup routes; the result is not a universal failure of static routing. Warm-up and restoration batches included transient losses. Fast Forward Time advanced the simulator, so the records do not support wall-clock convergence comparisons or a claim that OSPF was faster than RIP. Route selection, next hop and reachability establish the demonstrated benefit.

9.5 Critical Evaluation and Optimization

The strongest evidence comes from controlled changes paired with matching baselines. Delay calibration quantifies whether the fault behaves as configured. Bandwidth attainment shows both the useful lower-rate range and the limits of fine-grained local pacing. The timeout study demonstrates a real completion-versus-waiting tradeoff without pretending that a higher deadline optimizes the network.

The metrics also prevent misleading conclusions. Success counts accompany latency; unavailable successful timings are not zeros. Recovery confirmation includes the polling policy. Direct comparison measures interception overhead rather than assuming it is negligible. Routing metrics are interpreted within their protocol and are not numerically ranked across RIP and OSPF.

The current 1 KiB scheduling method could be evaluated against adaptive chunk sizes or batching at higher rates. A sound optimization study would hold the payload and target rate constant, compare attained goodput and CPU use, and retain cancellation/backpressure behavior. That is an engineering implication of the measured pacing limitation, not an implemented improvement claimed by this report.

No additional dashboard metric was required: successful timing, outcomes, goodput, baseline differences, raw observations and exports already support the study. The report adds repeated-run aggregation and calibration analysis without changing production behavior or the schema. This keeps the experiment mechanism consistent with the measured application version.

10. Results and Discussion

10.1 Findings and Inferences

First, added response time follows the configured delay, with the 1,500 ms condition producing 1,506.52 ms of paired added median. Second, payload pacing is close to its lower configured restrictions but attains only 61.81% at 128 KiB/s. Reporting both outcomes is more informative than assuming every rate is exact.

Third, the timeout intervention changes faulty completion by 100 percentage points while retaining the same 1,000 ms configured wait. The longer deadline tolerates that response; it does not remove its delay. Fourth, post-clearance observations supply repeated restoration evidence and distinguish service response timing from the duration of the recovery criterion.

Live results show why status, body size and completion must be considered together. A mock can complete with a small local JSON body; an injected 503 can complete with an error status; a stalled body can time out after headers. The cockpit therefore addresses manual application behavior that cannot be reduced to the workbench's fixed payload success criterion.

Packet evidence is consistent with real HTTP traffic under the tested conditions. The separately reviewed routed topology demonstrates alternate-path selection under a specific failure. The application and simulator answer related resilience questions at different layers; neither evidence set is used to invent measurements for the other.

10.2 Innovation Novelty and Evidence

The project's contribution is an inspectable local workflow: controlled fault settings, real requests, bounded execution, validated records, phase-specific analysis and a separate applied live configuration. Related tools establish proxy fault injection as prior art [3]. The contribution is integration and educational reproducibility, not a new routing protocol or a globally novel failure mechanism.

Useful improvement claims are attached to actual comparisons. The longer deadline improves completion for the chosen delayed workload, and dynamic route selection improves settled reachability over the no-backup control in the reviewed topology. Neither claim implies general superiority over unrelated systems. Calibration and interception results identify costs as well as benefits.

11. Conclusion

FaultNet implements controlled HTTP fault injection with saved experiments and a separate live development cockpit. A repeated evaluation of 100 workbench runs supports measurable delay, pacing, failure and restoration behavior. The timeout comparison improves observed completion from 0% to 100% by permitting longer waiting, while the high-rate pacing result exposes a concrete limitation. Packet analysis and reviewed routing observations provide complementary evidence. The system is useful for local development testing and networking education because configurations, measurements and explanations can be examined together.

12. Limitations and Future Enhancement

The study uses one shared Windows laptop, sequential requests and a fixed synthetic payload. Operating-system scheduling, background applications and capture activity can affect timing. Fifty faulty attempts per condition support descriptive comparisons, not universal availability or tail-latency guarantees. The bootstrap intervals describe variation among ten local blocks and do not remove environmental dependence.

One attempted run failed the record validator at the UTC-versus-elapsed recovery tolerance boundary. This is a persistence/validation limitation, separate from an injected connection failure. The rejected record remains in the local audit data and its count is included in record quality; only structurally validated records contribute to the comparison tables.

Request elapsed time is not TCP RTT, payload goodput is not wire bandwidth, and HTTP interruption is not random IP loss. Recovery confirmation depends on three-success sampling and its waits. The routing cases use small settled ping batches and simulated-time advancement; they establish path selection without a convergence-speed ranking. Representative interface screenshots illustrate the application rather than replacing the current measurement tables.

Further evaluation can vary payload sizes, use additional machines and controlled background load, compare pacing strategies at short chunk intervals, and measure routing convergence on a common time base. More frequent crash-durable saves and separate processes could be assessed alongside their storage and coordination costs. These extensions follow directly from the observed limits and do not alter the conclusions for the measured configuration.

References

[1] IETF. HTTP Semantics. RFC 9110.

https://www.rfc-editor.org/rfc/rfc9110

[2] Node.js. Performance measurement APIs and HTTP documentation.

https://nodejs.org/docs/latest-v22.x/api/perf_hooks.html

[3] Shopify. Toxiproxy official repository and documentation.

https://github.com/Shopify/toxiproxy

[4] Wireshark. User Guide, Packet Reassembly and Follow Protocol Streams.

https://www.wireshark.org/docs/wsug_html_chunked/ChAdvReassemblySection.html

[5] Malkin, G. RIP Version 2. RFC 2453.

https://www.rfc-editor.org/rfc/rfc2453

[6] Moy, J. OSPF Version 2. RFC 2328.

https://www.rfc-editor.org/rfc/rfc2328

[7] FaultNet. Source, implementation contracts and reproducible evaluation scripts.

https://github.com/Pranav-Suresh1024/FaultNet

Repository evidence: scripts/evaluate-report.mjs, scripts/analyze-report.py, docs/reports/evidence/summary.json, scenario-comparison.csv and packet-summary.json. Routing observations are documented in docs/packet-tracer-review.md. Raw captures and generated experiment records remain local; the shared aggregates and procedures identify how the reported results were derived.